Honeypot IDS – Pool test report

Generated: 2026-10-05 11:47:13
Target: ids (local) (https://localhost)
Exploits: A: CVE-2024-2387, B: CVE-2025-47577, C: GENERIC-WP-003, A: CVE-2024-1071, B: CVE-2025-4403, C: GENERIC-WP-002
Windows: 3

Result

EXCLUSIVE **Every attacker has a honeypot pool of their own**

- Attacker A → pool 1, Attacker B → pool 2, Attacker C → pool 3.

What is being tested

Attackers are told apart by session, not by address. Each test window has its own cookie jar and its own User-Agent and language, so the proxy sees three separate attackers. By default every malicious session is given a honeypot pool **of its own**: the router hands a new attacker a free ready pool no other session owns, and pool_manager builds a spare after each assignment. Two attackers only share a pool when none is free – growth is capped because the host is short of memory, CPU or disk, or every ready pool is already owned – and the router then reuses the ready pools in strict round-robin order.

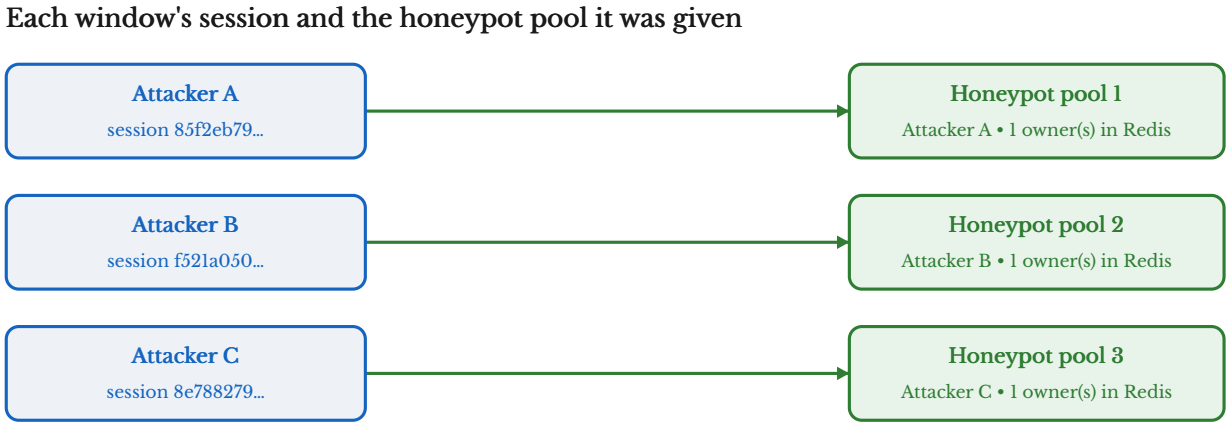


Figure 1. One arrow per test window, from its session to the honeypot pool the router bound it to. Green: the pool belongs to that window alone. Orange: shared under resource pressure (designed). Red: shared or merged without a reason. Grey: not bound.

Windows

Window	Session	Honeypot pool	Language	User-Agent (tail)
Attacker A	85f2eb79…	pool 1	en-US,en;q=0.9	6.11.2 Chrome/140.0.0.0 Safari/537.36 PoolTest/A
Attacker B	f521a050…	pool 2	de-DE,de;q=0.9	6.11.2 Chrome/140.0.0.0 Safari/537.36 PoolTest/B
Attacker C	8e788279…	pool 3	fr-FR,fr;q=0.9	6.11.2 Chrome/140.0.0.0 Safari/537.36 PoolTest/C

Exploits run per window

Each window runs a different exploit, picked from the ones a live probe showed divert within three attempts. If a chosen exploit still did not divert in three attempts, the next candidate is used instead.

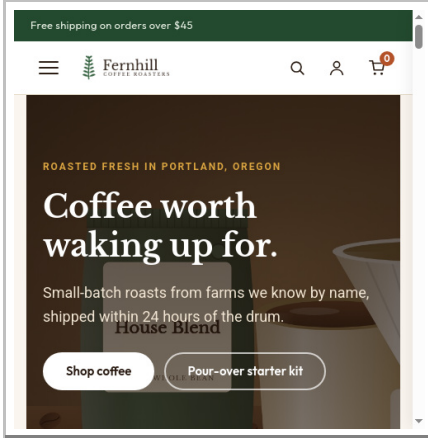
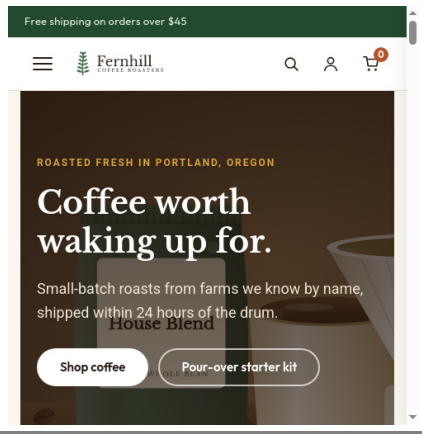
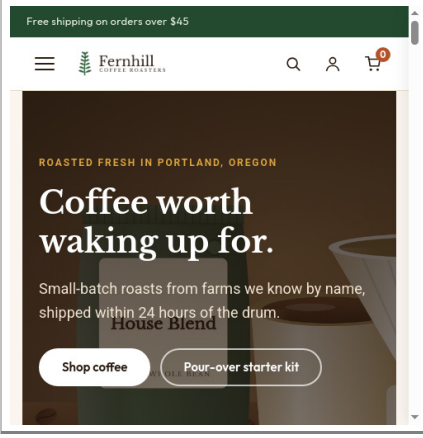
Window	Exploit	Name	Routing	Pool
A	CVE-2024-2387	Advanced Form Integration SQL injection	diverted after 1	pool 1
B	CVE-2025-47577	Gift Voucher file upload RCE	diverted after 1	pool 2
C	GENERIC-WP-003	User enumeration via ?author=	diverted after 1	pool 3
A	CVE-2024-1071	Ultimate Member SQL injection	diverted after 1	pool 1
B	CVE-2025-4403	Drag & Drop file upload MIME-type bypass	diverted after 1	pool 2
C	GENERIC-WP-002	Installer file access	diverted after 1	pool 3

Step by step

All three windows move in every step. An attacker's pool must appear in their own step, differ from the earlier ones, and never change afterwards.

Step 1: All windows open the shop

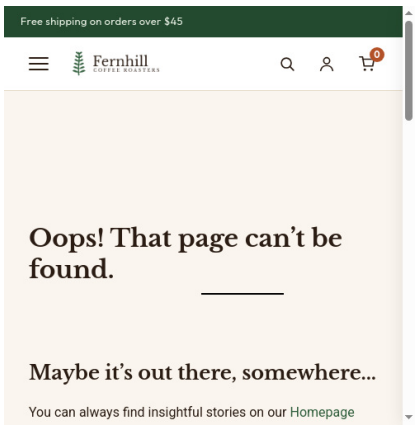
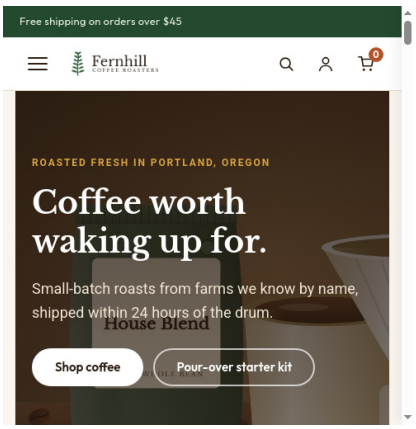
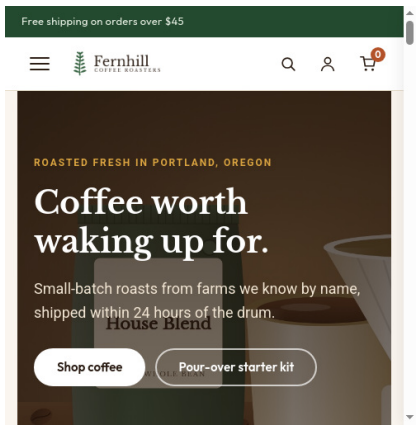
Three new sessions, all routed to production.

Attacker A	Attacker B	Attacker C
Browses the shop	Browses the shop	Browses the shop
PRODUCTION no pool • score 0	PRODUCTION no pool • score 0	PRODUCTION no pool • score 0
		

Step 2: Attacker A attacks with CVE-2024-2387

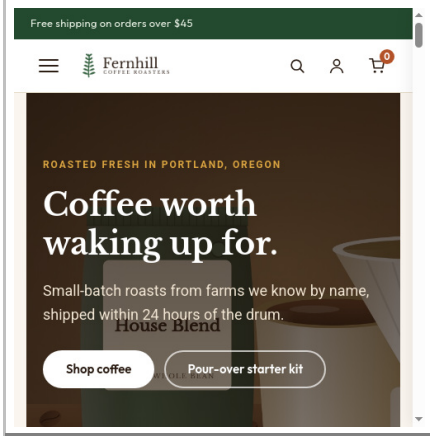
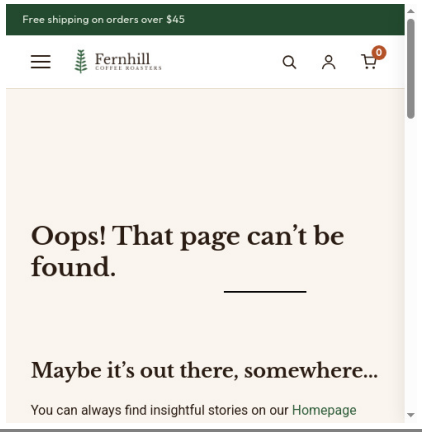
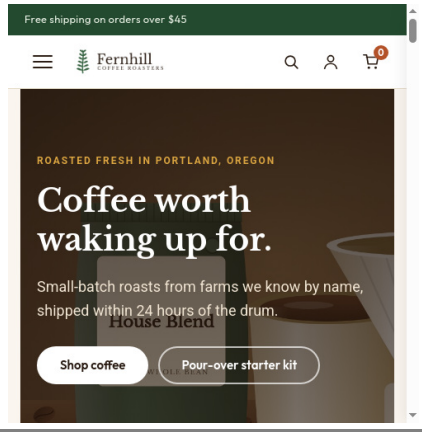
Advanced Form Integration SQL injection. The session must be bound to a pool no other window has, and the others must stay on production.

Attacker A	Attacker B	Attacker C
Opens CVE-2024-2387 (1 attempt(s))	Browses the shop	Browses the shop

HONEYPOT pool 1 • score 100	PRODUCTION no pool • score 0	PRODUCTION no pool • score 0
		

Step 3: Attacker B attacks with CVE-2025-47577

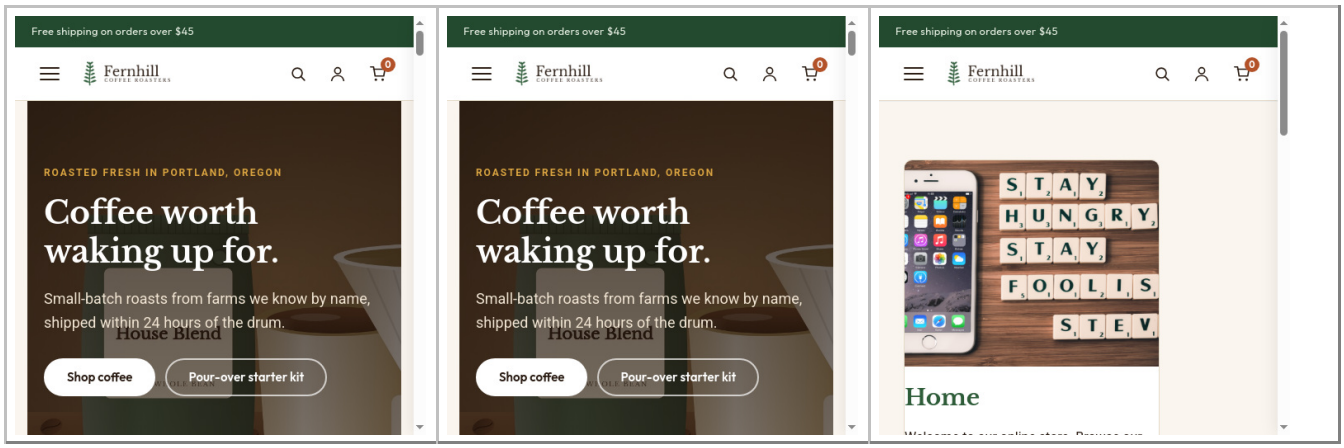
Gift Voucher file upload RCE. The session must be bound to a pool no other window has, and the others must stay on production.

Attacker A	Attacker B	Attacker C
Browses the shop	Opens CVE-2025-47577 (1 attempt(s))	Browses the shop
HONEYPOT pool 1 • score 100	HONEYPOT pool 2 • score 95	PRODUCTION no pool • score 0
		

Step 4: Attacker C attacks with GENERIC-WP-003

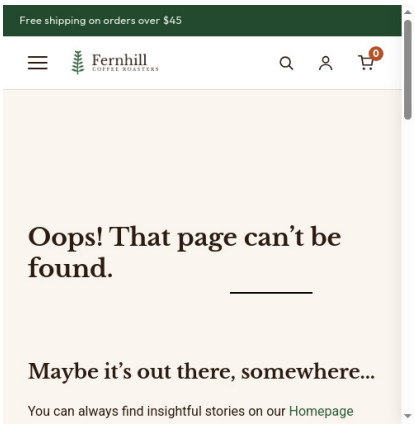
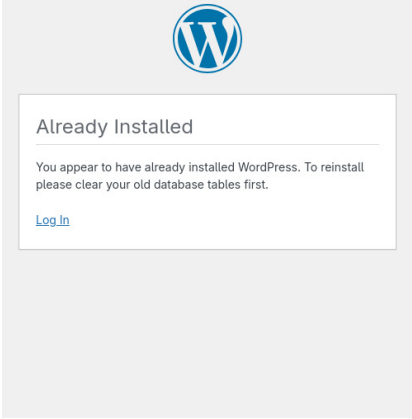
User enumeration via ?author=. The session must be bound to a pool no other window has, and the others must stay on production.

Attacker A	Attacker B	Attacker C
Browses the shop	Browses the shop	Opens GENERIC-WP-003 (1 attempt(s))
HONEYPOT pool 1 • score 100	HONEYPOT pool 2 • score 100	HONEYPOT pool 3 • score 100



Step 5: All windows run a second exploit

Sticky binding: each second exploit must land in the pool the window already owns, nobody changes pool, nobody is merged.

Attacker A	Attacker B	Attacker C
Opens CVE-2024-1071	Opens CVE-2025-4403	Opens GENERIC-WP-002
HONEYPOT pool 1 • score 100	HONEYPOT pool 2 • score 100	HONEYPOT pool 3 • score 100
0		

What each honeypot container saw and changed

Read from each pool's own eshop and database containers since the run started (2026-10-05T09:46:50Z). Requests are matched to windows by their User-Agent tag, so a pool should show its owner's exploits and nobody else's. A pool no window owns is shown for contrast.

pool 1 window A

Windows seen	A (4 requests)
Requests received	• A GET /?page=advanced-form-integration-log&integration_id=1%27%20UNION%20SELECT%20user_login%2Cu → 404 • A GET / → 200 • A GET /wp-admin/admin-ajax.php?action=um_get_members&directory_id=1%27%20UNION%20SELECT%20user_l → 400
Files changed	• no files written

Database activity	questions +976, select +923, insert +4, update +4, delete +4

pool 2 window B

Windows seen	B (3 requests)
Requests received	• **B** GET /?action=mwb_wgm_preview_mail&attachment=shell.php → 404 • **B** GET / → 200 • **B** GET /?action=dnd-wc-upload-file&name=shell.php → 404
Files changed	• no files written
Database activity	questions +793, select +750, insert +4, update +1, delete +4

pool 3 window C

Windows seen	C (3 requests)
Requests received	• **C** GET /?author=1 → 301 • **C** GET /author/admin/ → 200 • **C** GET /wp-admin/install.php → 200
Files changed	• no files written
Database activity	questions +618, select +582, insert +2, update +1, delete +2

Pool registry at export time

Growth capped – low memory: 1110 MB available, need 2560 MB. New attackers reuse the existing pools round-robin instead of getting a new one.
Ready pools: 3 (1, 2, 3). Free (unowned) spares: 0. Attackers per pool: pool 1: 1, pool 2: 1, pool 3: 1. Round-robin reuse assignments so far: 3.